Deep Neural Networks

How Layers Learn to Recognize Complex Patterns

Dr. Nikhilanand Arya

Indian Institute of Information Technology Allahabad

How can a machine learn from its mistakes?

An interactive journey into linear & Non-linear classification

What Does a Neural Network Learn?

A neural network receives an input and tries to produce the correct output.



The Learning Goal

Find the weights that transform inputs into correct predictions.

The Goal: Learn a Decision Boundary

Given an input $\mathbf{x}$, the Perceptron calculates:

$$z = \mathbf{w}^T \mathbf{x}$$

Then makes a decision:

$$y = f(\mathbf{w}^T \mathbf{x})$$



Where Does Learning Begin?

Before learning anything, the Perceptron has to start somewhere.

Initial Weights
A small random guess

For example:

$$\mathbf{w} = \begin{bmatrix} 0.12 \\ -0.05 \end{bmatrix}$$

Key Idea

The Perceptron does not know the correct decision boundary yet. It starts with a guess and improves it through experience.

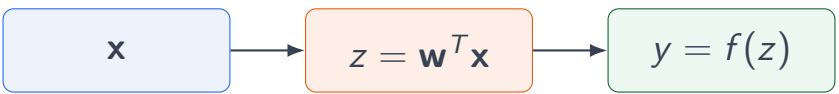
How Does the Perceptron Make a Decision?

First, combine the input using the weights:

$$z = \mathbf{w}^T \mathbf{x}$$

Then convert the result into a class label:

$$y = f(z)$$



How Should the Weights Change?

$$w_i(t+1) = w_i(t) + \alpha(d_j - y_j)x_{j,i}$$

New weight

Old Weight

Correction from error

Read the Equation Like This

New Weight = Old Weight + Correction

What Does "Correction from Error" Mean?

$$w_i(t+1) = w_i(t) + \underbrace{\alpha(d_j - y_j)x_{j,i}}_{\text{Correction from error}}$$

Learning Rate
α
How big a step?

Error
$d_j - y_j$
Which direction?

Input
$x_{j,i}$
How much influence?

Correction

$$\alpha(d_j - y_j)x_{j,i}$$

determines **how much** and **in which direction** each weight should change.

Reading the Update Rule

$$\boxed{\text{New Weight} = \text{Old Weight} + \text{Correction}}$$

$$\boxed{\text{Correction} = \underbrace{\alpha}_{\text{How much}} \times \underbrace{(d_j - y_j)}_{\text{Which direction}} \times \underbrace{x_{j,i}}_{\text{Which feature}}}$$

In Plain English

If the prediction is wrong, adjust each weight in a direction that helps the Perceptron make a better prediction next time.

Perceptron Example: Can It Learn AND?

Instance	x_1	x_2	Target (d)
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1

Starting Point

$$w_1 = 1.2, \qquad w_2 = 0.6, \qquad \theta = 1, \qquad \alpha = 0.5$$

Challenge

Will the Perceptron classify all four AND examples correctly?

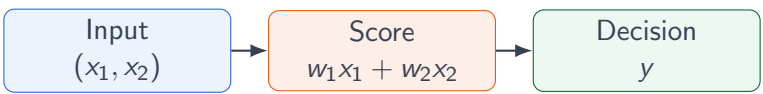
How Will the Perceptron Decide?

First calculate the weighted score:

$$z = w_1 x_1 + w_2 x_2$$

Then compare it with the threshold:

$$y = \begin{cases} 1, & z \geq \theta \\ 0, & z < \theta \end{cases}$$



Epoch 1: Instance 1

$$x = (0, 0), \qquad d = 0$$

$$z = 1.2(0) + 0.6(0) = 0$$

$$0 < 1 \quad \Rightarrow \quad y = 0$$

Check

Prediction: $y = 0$ **Target:** $d = 0$

Correct! No update needed.

Epoch 1: Instance 2

$$x = (0, 1), \qquad d = 0$$

$$z = 1.2(0) + 0.6(1) = 0.6$$

$$0.6 < 1 \quad \Rightarrow \quad y = 0$$

Check

Prediction: $y = 0$ **Target:** $d = 0$

Correct! No update needed.

Epoch 1: Instance 3

$$x = (1, 0), \qquad d = 0$$

$$z = 1.2(1) + 0.6(0) = 1.2$$

$$1.2 \geq 1 \quad \Rightarrow \quad y = 1$$

Something Went Wrong!

Prediction: $y = 1$ and $y \neq d$
The Perceptron made a mistake.
Time to learn.

How Should the Perceptron Correct This Mistake?

The update rule is:

$$w_i^{new} = w_i^{old} + \underbrace{\alpha(d-y)x_i}_{\text{Correction}}$$

$$\alpha = 0.5, \qquad d - y = 0 - 1 = -1$$

$$\text{Correction} = 0.5(-1)x_i$$

Think

The prediction was too high.
The error is negative.
Which weight should actually change?

Updating the Weights

$$w_i^{new} = w_i^{old} + \alpha(d - y)x_i$$

For $x = (1, 0)$:

$$w_1 = 1.2 + 0.5(0 - 1)(1) = 0.7$$

$$w_2 = 0.6 + 0.5(0 - 1)(0) = 0.6$$

New Weights

$$\boxed{w_1 = 0.7, \qquad w_2 = 0.6}$$

Epoch 1: Instance 4

$$w_1 = 0.7, \qquad w_2 = 0.6$$

$$x = (1, 1), \qquad d = 1$$

$$z = 0.7(1) + 0.6(1) = 1.3$$

$$1.3 \geq 1 \quad \Rightarrow \quad y = 1$$

Check

Correct! No update needed.

End of Epoch 1

Instance	Prediction	Update?
1	Correct	No
2	Correct	No
3	Wrong	Yes
4	Correct	No

Weights After Epoch 1

$$\boxed{w = (0.7, 0.6)}$$

What Did the Perceptron Learn?

Only one mistake was enough to move the decision boundary.

Epoch 2: Does It Still Make Mistakes?

$w_1 = 0.7, \quad w_2 = 0.6, \quad \theta = 1$

Instance	x_1, x_2	$z = w^T x$	y	d
1	$(0, 0)$	0	0	0
2	$(0, 1)$	0.6	0	0
3	$(1, 0)$	0.7	0	0
4	$(1, 1)$	1.3	1	1

Result

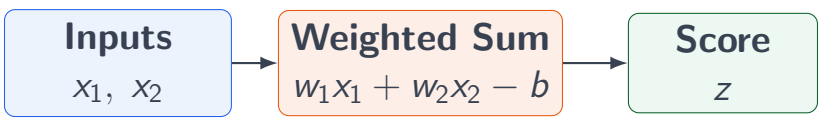
No mistakes in this epoch.

The Perceptron has converged.

What Does a Perceptron Compute?

First, combine the input features using their weights:

$$z = w_1 x_1 + w_2 x_2 - b$$



Key Idea

The weights decide how strongly each input influences the final decision.

From Score to Decision

$$z = w_1 x_1 + w_2 x_2 - b$$

$$y = f(z)$$

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Equivalent Decision Rule

$$y = \begin{cases} 1, & w^T x \geq b \\ 0, & w^T x < b \end{cases}$$

Where Does the Decision Change?

The decision changes exactly at:

$$z = 0$$

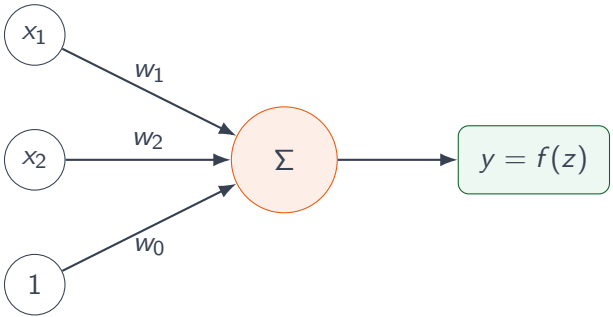
$$z = w_1 x_1 + w_2 x_2 - b$$

the boundary is:

$$\boxed{w_1 x_1 + w_2 x_2 - b = 0}$$

On one side, the Perceptron predicts Class 0.
On the other side, it predicts Class 1.

The Perceptron as a Neural Unit



$$z = w_1 x_1 + w_2 x_2 + w_0(1)$$

Bias as a Weight

The bias can be treated as a weight connected to a constant input of 1.

What Do the Weights Mean Geometrically?

For two input features, the decision boundary is:

$$w_1 x_1 + w_2 x_2 - b = 0$$



Building a Perceptron in Python

We will translate the learning process into code.



Our Goal

Turn the Perceptron learning algorithm into a reusable Python class.

Step 1: Define the Decision Rule

```
import numpy as np

def unit_step_func(x):
    return np.where(x >= 0, 1, 0)
```

What Does This Function Do?

It converts a numerical score into a class label.

$$x \geq 0 \Rightarrow 1 \qquad x < 0 \Rightarrow 0$$

Connection to the Perceptron

The weighted sum produces a score. The step function converts that score into a decision.

Step 2: Create the Perceptron

```
class Perceptron:

    def __init__(self, learning_rate=0.01,
                 n_iters=100):

        self.lr = learning_rate
        self.n_iters = n_iters

        self.weights = None
        self.bias = None
```

Step 3: Start With No Knowledge

```
def fit(self, X, y):

    n_samples, n_features = X.shape

    self.weights = np.zeros(n_features)
    self.bias = 0
```

Initially:

$$\mathbf{w} = (0, 0, \ldots, 0) \qquad b = 0$$

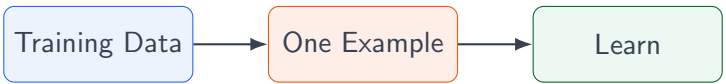
Key Idea

The Perceptron starts with no learned preference. Training gradually changes the weights and bias.

Step 4: Learn From the Data Repeatedly

```
for _ in range(self.n_iters):

    for idx, x_i in enumerate(X):
```



Epoch

One epoch means the Perceptron has seen every training example once.

Step 5: Make a Prediction

```
linear_output = (
    np.dot(x_i, self.weights)
    + self.bias
)

y_predicted = unit_step_func(linear_output)
```

$$z = \mathbf{x}_i^T \mathbf{w} + b$$

$$y = f(z)$$

What Happens Here?

Input $\rightarrow$ Weighted Sum $\rightarrow$ Activation Function $\rightarrow$ Prediction

Step 6: Learn From the Mistake

```
update = self.lr * (
    y[idx] - y_predicted
)

self.weights += update * x_i
self.bias += update
```

$$\Delta w_i = \alpha(d - y)x_i$$

$$\Delta b = \alpha(d - y)$$

Read the Code

New Parameters = Old Parameters + Correction

Putting the Training Process Together

```
def fit(self, X, y):

    n_samples, n_features = X.shape

    self.weights = np.zeros(n_features)
    self.bias = 0

    for _ in range(self.n_iters):

        for idx, x_i in enumerate(X):

            linear_output = (
                np.dot(x_i, self.weights)
                + self.bias
            )
            .
```

Putting the Training Process Together

```
def fit(self, X, y):

            .
            .
            .

            y_predicted = unit_step_func(
                linear_output
            )

            update = self.lr * (
                y[idx] - y_predicted
            )

            self.weights += update * x_i
            self.bias += update
```

Step 7: Use the Trained Model

```
def predict(self, X):

    linear_output = (
        np.dot(X, self.weights)
        + self.bias
    )

    return unit_step_func(linear_output)
```

New Input → Weighted Sum → Step Function → Class

Output

The trained Perceptron returns a predicted class: 0 or 1

A Simple Problem for Our Perceptron

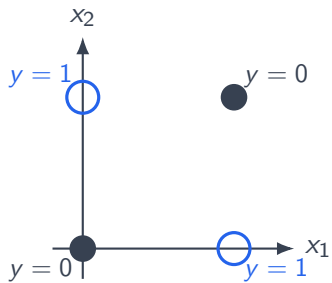
Can our Perceptron learn this rule?

	x_1	x_2	
	x_1	x_2	**XOR**
	0	0	0
	0	1	1
	1	0	1
	1	1	0

The XOR Rule

Output is 1 when the two inputs are different.

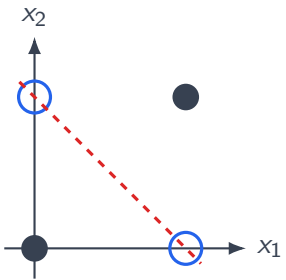
Let's Plot the XOR Data



Your Turn

Can you draw one straight line that separates the two classes?

Try to Separate the Classes



What Happens?

A line may separate some points, but it will always leave at least one point on the wrong side.

The Problem: One Line Is Not Enough

XOR cannot be separated by one straight line.

But a single Perceptron can only create:

$$w_1 x_1 + w_2 x_2 + b = 0$$

which is a straight line in two dimensions.

Therefore

XOR is **not linearly separable**.

Can a Perceptron Satisfy XOR?

For XOR, assume:

$$y = \begin{cases} 1, & w_1 x_1 + w_2 x_2 + b \geq 0 \\ 0, & w_1 x_1 + w_2 x_2 + b < 0 \end{cases}$$

The point $(0, 0)$ belongs to Class 0:

$$b < 0$$

The point $(1, 0)$ belongs to Class 1:

$$w_1 + b \geq 0$$

$$w_1 \geq -b$$

More Constraints from XOR

For $(0, 1)$, the output must be 1:

$$w_2 + b \geq 0$$

$$w_2 \geq -b$$

Adding the two inequalities:

$$w_1 + w_2 \geq -2b$$

Since:

$$b < 0$$

$$w_1 + w_2 + b > 0$$

The Contradiction

For the input:

$$(1, 1)$$

XOR requires:

$$w_1 + w_2 + b < 0$$

But from the other XOR constraints, we obtained:

$$w_1 + w_2 + b > 0$$

Impossible!

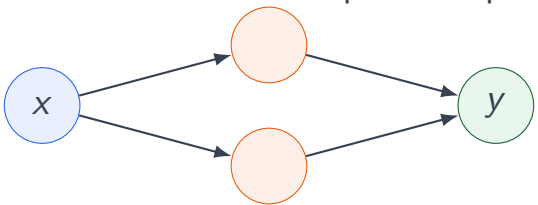
The same expression cannot be both: > 0 and < 0

Therefore, no single Perceptron can solve XOR.

So... How Do We Solve XOR?

If one Perceptron is not enough...

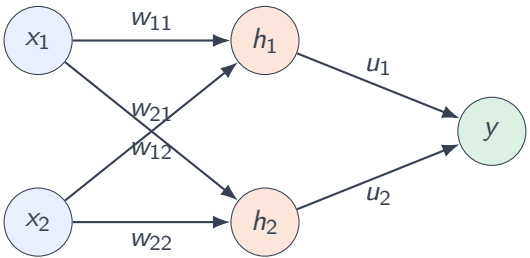
What if we combine multiple Perceptrons?



New Idea

A hidden layer can combine multiple linear decisions to create a non-linear decision boundary.

Every Connection Has Its Own Weight



How Does Information Flow Through the Network?

Hidden Neuron h_1

$$h_1 = f(w_{11}x_1 + w_{12}x_2 + b_1)$$

Hidden Neuron h_2

$$h_2 = f(w_{21}x_1 + w_{22}x_2 + b_2)$$

Output Neuron

$$y = f(u_1h_1 + u_2h_2 + b_3)$$

The Key Idea: Create New Representations

A single Perceptron sees the original inputs:

$$(x_1, x_2)$$

A hidden layer can create new features:

$$h_1 = f(w_{11}x_1 + w_{12}x_2 + b_1)$$

$$h_2 = f(w_{21}x_1 + w_{22}x_2 + b_2)$$

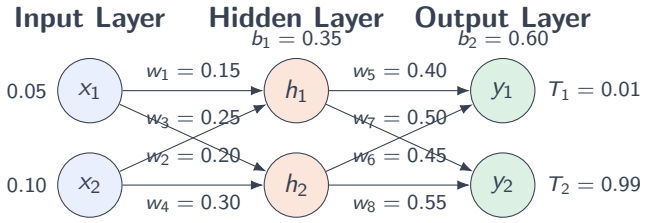
The output neuron then combines them:

$$y = f(u_1h_1 + u_2h_2 + b)$$

The Big Idea

The network learns which representation of the data makes the problem easier to solve.

Our Neural Network: Complete Example



Complete Setup

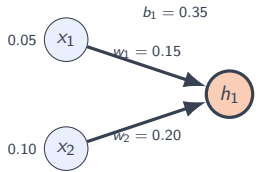
Inputs: $x_1 = 0.05, \quad x_2 = 0.10$

Biases: $b_1 = 0.35, \quad b_2 = 0.60$

Activation: $\sigma(z) = \frac{1}{1+e^{-z}}$

Targets: $T_1 = 0.01, \quad T_2 = 0.99$ **Learning Rate:** $\eta = 0.5$

Forward Pass: Activating h_1



We are computing h_1

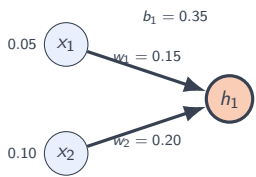
Step 1: Weighted Sum

$$net_{h_1} = w_1 x_1 + w_2 x_2 + b_1$$

$$= (0.15)(0.05) + (0.20)(0.10) + 0.35$$

$$\boxed{net_{h_1} = 0.3775}$$

Forward Pass: Activating h_1



We are computing h_1

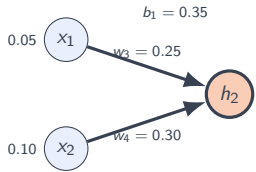
Step 2: Activate

$$h_1 = \sigma(net_{h_1})$$

$$h_1 = \frac{1}{1 + e^{-0.3775}}$$

$$\boxed{h_1 \approx 0.5932}$$

Forward Pass: Activating h_2



Now we compute h_2

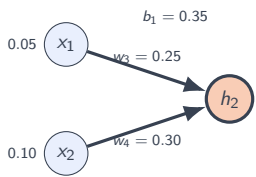
Step 1: Weighted Sum

$$net_{h_2} = w_3 x_1 + w_4 x_2 + b_1$$

$$= (0.25)(0.05) + (0.30)(0.10) + 0.35$$

$$\boxed{net_{h_2} = 0.3925}$$

Forward Pass: Activating h_2



Now we compute h_2

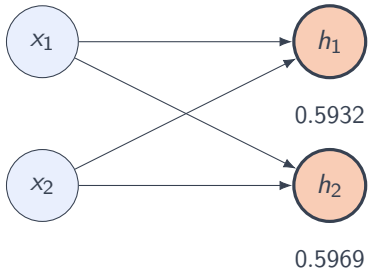
Step 2: Activate

$$h_2 = \sigma(net_{h_2})$$

$$h_2 = \frac{1}{1 + e^{-0.3925}}$$

$$\boxed{h_2 \approx 0.5969}$$

The Hidden Layer Is Now Activated



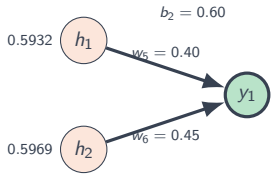
New Representation

$(x_1, x_2) = (0.05, 0.10)$ has been transformed into $\boxed{(h_1, h_2) = (0.5932, 0.5969)}$

Next

These hidden outputs now become inputs to the output layer.

Forward Pass: Activating y_1



We are computing y_1

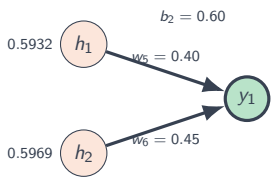
Step 1: Weighted Sum

$$net_{y_1} = w_5 h_1 + w_6 h_2 + b_2$$

$$= (0.40)(0.5932) + (0.45)(0.5969) + 0.60$$

$$\boxed{net_{y_1} \approx 1.106}$$

Forward Pass: Activating y_1



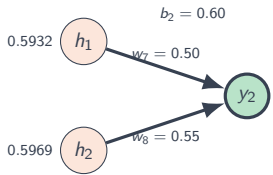
We are computing y_1

Step 2: Activate

$$y_1 = \sigma(1.106)$$

$$\boxed{y_1 \approx 0.7513}$$

Forward Pass: Activating y_2



Now we compute y_2

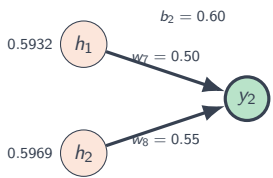
Step 1: Weighted Sum

$$net_{y_2} = w_7 h_1 + w_8 h_2 + b_2$$

$$= (0.50)(0.5932) + (0.55)(0.5969) + 0.60$$

$$\boxed{net_{y_2} \approx 1.225}$$

Forward Pass: Activating y_2



Now we compute y_2

Step 2: Activate

$$y_2 = \sigma(1.225)$$

$$\boxed{y_2 \approx 0.7729}$$

Did the Network Make the Right Prediction?

Output	Network Prediction	Target
y_1	0.7513	0.01
y_2	0.7729	0.99

The Network's Prediction

$$\boxed{(y_1, y_2) = (0.7513, 0.7729)}$$

But We Wanted

$$\boxed{(T_1, T_2) = (0.01, 0.99)}$$

The prediction is not correct enough.

How Wrong Is the Network?

For each output, we measure the squared difference:

$$E_i = \frac{1}{2}(T_i - y_i)^2$$

Total Error

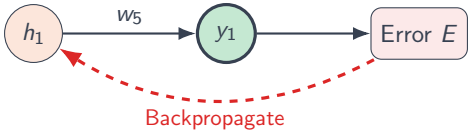
$$E = \frac{1}{2}(T_1 - y_1)^2 + \frac{1}{2}(T_2 - y_2)^2$$

$$= \frac{1}{2}(0.01 - 0.7513)^2 + \frac{1}{2}(0.99 - 0.7729)^2 \approx \boxed{0.298}$$

Next Question

How can we change the weights to reduce this error?

Backpropagation: Where Did the Error Come From?



Goal

Find out how much the error changes when we change a weight:

$$\boxed{\frac{\partial E}{\partial w_5}}$$

Big Idea

Move **backwards**: Error → Output → Weight

The Chain Rule: Follow the Path Backwards

To understand how w_5 affects the error, follow this path:

$$w_5 \rightarrow net_{y_1} \rightarrow y_1 \rightarrow E$$

Therefore,

$$\boxed{\frac{\partial E}{\partial w_5} = \frac{\partial E}{\partial y_1} \cdot \frac{\partial y_1}{\partial net_{y_1}} \cdot \frac{\partial net_{y_1}}{\partial w_5}}$$

1. Error

How does y_1 change the error?

2. Activation

How does net_{y_1} change y_1?

3. Weight

How does w_5 change net_{y_1}?

Step 1: How Does y_1 Affect the Error?



Prediction $\rightarrow$ Error

Loss Function

$$E = \frac{1}{2}(T_1 - y_1)^2$$

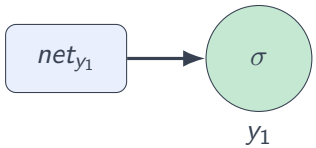
Derivative

$$\frac{\partial E}{\partial y_1} = y_1 - T_1$$

Our Value

$$0.7513 - 0.01 = \boxed{0.7413}$$

Step 2: How Does net_{y_1} Affect y_1?



Net input $\rightarrow$ Sigmoid $\rightarrow$ Output

Activation

$$y_1 = \sigma(net_{y_1})$$

Sigmoid Derivative

$$\frac{\partial y_1}{\partial net_{y_1}} = y_1(1 - y_1)$$

Our Value

$$0.7513(1 - 0.7513) \approx \boxed{0.1869}$$

Where Does $\sigma(z)(1-\sigma(z))$ Come From?

Start with:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Rewrite:

$$\sigma(z) = (1+e^{-z})^{-1}$$

Differentiate:

$$\frac{d\sigma}{dz} = -(1+e^{-z})^{-2}(-e^{-z})$$

$$= \frac{e^{-z}}{(1+e^{-z})^2}$$

Where Does $\sigma(z)(1-\sigma(z))$ Come From?

Now observe:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

and

$$1-\sigma(z) = \frac{e^{-z}}{1+e^{-z}}$$

Therefore:

$$\boxed{\frac{d\sigma}{dz} = \sigma(z)(1-\sigma(z))}$$

Activation Functions and Their Derivatives

Activation	Function	Derivative
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma'(x) = \sigma(x)(1-\sigma(x))$
Tanh	$\tanh(x)$	$1 - \tanh^2(x)$
ReLU	$\max(0, x)$	$\begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases}$
Leaky ReLU	$\max(\alpha x, x)$	$\begin{cases} 1, & x > 0 \\ \alpha, & x < 0 \end{cases}$
Linear	$f(x) = x$	$f'(x) = 1$

Why Do We Care About the Derivative?

The derivative determines **how strongly a neuron passes error backward**.

Step 3: How Does w_5 Affect net_{y_1}?



h_1 is multiplied by w_5

Net Input

$$net_{y_1} = w_5 h_1 + w_6 h_2 + b_2$$

When differentiating with respect to w_5:

$$\boxed{\frac{\partial net_{y_1}}{\partial w_5} = h_1}$$

Our Value

$$\boxed{h_1 = 0.5932}$$

Combine the Three Steps

$$\boxed{\frac{\partial E}{\partial w_5}} = \underbrace{\frac{\partial E}{\partial y_1}}_{\text{Error}} \cdot \underbrace{\frac{\partial y_1}{\partial net_{y_1}}}_{\text{Sigmoid}} \cdot \underbrace{\frac{\partial net_{y_1}}{\partial w_5}}_{\text{Connection}}$$

Part	Value
Error term	0.7413
Sigmoid derivative	0.1869
Input to weight w_5	0.5932

$$\frac{\partial E}{\partial w_5} = (0.7413)(0.1869)(0.5932)$$

$$\boxed{\frac{\partial E}{\partial w_5} \approx 0.0822}$$

What Does This Gradient Tell Us?

$$\boxed{\frac{\partial E}{\partial w_5} \approx 0.0822}$$

Interpretation

- The gradient is **positive**.
- Increasing w_5 would increase the error.
- Therefore, to reduce error, move w_5 in the opposite direction.

$$\boxed{w_5^{new} = w_5^{old} - \eta \frac{\partial E}{\partial w_5}}$$

Gradient tells us which direction increases error

Gradient for w_6: Follow the Connection



We are updating this connection.

Path of Influence

$$w_6 \rightarrow net_{y_1} \rightarrow y_1 \rightarrow E$$

Therefore,

$$\frac{\partial E}{\partial w_6} = \frac{\partial E}{\partial y_1} \cdot \frac{\partial y_1}{\partial net_{y_1}} \cdot \frac{\partial net_{y_1}}{\partial w_6}$$

Computing the Gradient for w_6

The Three Components

$$\frac{\partial E}{\partial y_1} = y_1 - T_1 = 0.7513 - 0.01 = 0.7413$$

$$\frac{\partial y_1}{\partial net_{y_1}} = y_1(1 - y_1) \approx 0.1869$$

$$\frac{\partial net_{y_1}}{\partial w_6} = h_2 = 0.5968$$

Important Difference

For w_5, the connected input was h_1.

For w_6, the connected input is h_2.

Final Gradient for w_6

Combine the Three Terms

$$\frac{\partial E}{\partial w_6} = (y_1 - T_1) \cdot y_1(1 - y_1) \cdot h_2$$

$$= (0.7413)(0.1869)(0.5968)$$

$$\boxed{\frac{\partial E}{\partial w_6} \approx 0.0826}$$

Key Insight

Both w_5 and w_6 affect the same output neuron y_1.

The difference is the signal entering each connection:

Gradient for w_7: Which Path Are We Following?



We update the connection from h_1 to y_2.

Path of Influence

$$w_7 \rightarrow net_{y_2} \rightarrow y_2 \rightarrow E$$

Therefore,

$$\frac{\partial E}{\partial w_7} = \frac{\partial E}{\partial y_2} \cdot \frac{\partial y_2}{\partial net_{y_2}} \cdot \frac{\partial net_{y_2}}{\partial w_7}$$

Computing the Gradient for w_7

Step 1: Output Error

$$\frac{\partial E}{\partial y_2} = y_2 - T_2$$

$$= 0.773 - 0.99 = \boxed{-0.217}$$

Step 2: Sigmoid Sensitivity

$$\frac{\partial y_2}{\partial net_{y_2}} = y_2(1 - y_2)$$

$$= 0.773(1 - 0.773) \approx \boxed{0.1755}$$

Computing the Gradient for w_7

Step 3: Effect of w_7

$$\frac{\partial net_{y_2}}{\partial w_7} = h_1 = \boxed{0.5932}$$

Final Gradient for w_7

Combine the Three Terms

$$\frac{\partial E}{\partial w_7} = (y_2 - T_2) \cdot y_2(1 - y_2) \cdot h_1$$

$$= (-0.217)(0.1755)(0.5932)$$

$$\boxed{\frac{\partial E}{\partial w_7} \approx -0.0226}$$

Negative Gradient Means increasing w_7 will reduce the error.

Gradient for w_8: Follow the Connection



We update the connection from h_2 to y_2.

Path of Influence

$$w_8 \rightarrow net_{y_2} \rightarrow y_2 \rightarrow E$$

Therefore,

$$\frac{\partial E}{\partial w_8} = \frac{\partial E}{\partial y_2} \cdot \frac{\partial y_2}{\partial net_{y_2}} \cdot \frac{\partial net_{y_2}}{\partial w_8}$$

Computing the Gradient for w_8

Step 1: Output Error

$$\frac{\partial E}{\partial y_2} = y_2 - T_2 = 0.773 - 0.99 = \boxed{-0.217}$$

Step 2: Sigmoid Sensitivity

$$\frac{\partial y_2}{\partial net_{y_2}} = y_2(1 - y_2)$$

$$= 0.773(1 - 0.773) \approx \boxed{0.1755}$$

Computing the Gradient for w_8

Step 3: Effect of w_8

$$\frac{\partial net_{y_2}}{\partial w_8} = h_2 = \boxed{0.5968}$$

Final Gradient for w_8

Combine the Three Terms

$$\frac{\partial E}{\partial w_8} = (y_2 - T_2) \cdot y_2(1 - y_2) \cdot h_2$$

$$= (-0.217)(0.1755)(0.5968)$$

$$\boxed{\frac{\partial E}{\partial w_8} \approx -0.0227}$$

Key Insight

w_7 and w_8 affect the same output y_2.

The only difference is the hidden neuron connected to them:

A Pattern Is Emerging

Weight	Output	Input
w_5	y_1	h_1
w_6	y_1	h_2
w_7	y_2	h_1
w_8	y_2	h_2

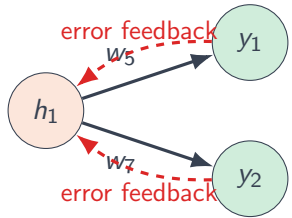
For every output-layer weight:

$$\frac{\partial E}{\partial w} = \underbrace{(y - T)}_{\text{Output Error}} \cdot \underbrace{y(1-y)}_{\text{Activation Sensitivity}} \cdot \underbrace{\text{Input}}_{\text{Signal on the Connection}}$$

Think

Once we update w_5, w_6, w_7, w_8, how do we send the error back to the hidden layer?

Backpropagation to the Hidden Layer



Key Observation

h_1 contributes to **both** y_1 and y_2. Therefore, its total error must include feedback from both outputs.

$$\frac{\partial E}{\partial h_1} = \text{feedback from } y_1 + \text{feedback from } y_2$$

How Does Error Reach h_1?

There are two paths:

$$h_1 \to y_1 \to E$$

and

$$h_1 \to y_2 \to E$$

Therefore,

$$\frac{\partial E}{\partial h_1} = \underbrace{\frac{\partial E}{\partial y_1}\frac{\partial y_1}{\partial net_{y_1}}\frac{\partial net_{y_1}}{\partial h_1}}_{\text{Path through } y_1} + \underbrace{\frac{\partial E}{\partial y_2}\frac{\partial y_2}{\partial net_{y_2}}\frac{\partial net_{y_2}}{\partial h_1}}_{\text{Path through } y_2}$$

Important

When one neuron affects multiple downstream neurons, the error contributions from all paths are added.

The Error Signal Arriving at h_1

Contribution from y_1

$$(y_1 - T_1)\, y_1(1 - y_1)\, w_5$$

Contribution from y_2

$$(y_2 - T_2)\, y_2(1 - y_2)\, w_7$$

Therefore,

$$\boxed{\frac{\partial E}{\partial h_1} = (y_1 - T_1)y_1(1 - y_1)w_5 + (y_2 - T_2)y_2(1 - y_2)w_7}$$

Computing the Error Signal at h_1

Contribution Through y_1

$$(0.7513 - 0.01)(0.7513)(1 - 0.7513)(0.40) \approx 0.0555$$

Contribution Through y_2

$$(0.773 - 0.99)(0.773)(1 - 0.773)(0.50) \approx -0.0190$$

Therefore,

$$\boxed{\frac{\partial E}{\partial h_1} \approx 0.0555 - 0.0190 = 0.0365}$$

Before Updating w_1: Pass Through the Hidden Neuron

The error has reached h_1:

$$\frac{\partial E}{\partial h_1} \approx 0.0365$$

But h_1 is the output of a sigmoid neuron:

$$h_1 = \sigma(net_{h_1})$$

Therefore, we must account for the sensitivity of the hidden neuron:

$$\frac{\partial h_1}{\partial net_{h_1}} = h_1(1 - h_1)$$

$$= 0.5932(1 - 0.5932) \approx 0.241$$

Gradient for w_1: Follow the Path



x_1 enters h_1 through w_1.

The path is:

$$w_1 \to net_{h_1} \to h_1 \to E$$

Therefore,

$$\boxed{\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial h_1} \frac{\partial h_1}{\partial net_{h_1}} \frac{\partial net_{h_1}}{\partial w_1}}$$

Why Does x_1 Appear in the Gradient?

The hidden neuron's net input is:

$$net_{h_1} = w_1 x_1 + w_2 x_2 + b_1$$

We differentiate with respect to w_1:

$$\boxed{\frac{\partial net_{h_1}}{\partial w_1} = x_1}$$

In our example:

$$\boxed{x_1 = 0.05}$$

Pattern

The gradient of a weight always includes the **input entering that weight**.

Computing the Gradient for w_1

We now have all three pieces:

$$\frac{\partial E}{\partial h_1} \approx 0.0365$$

$$\frac{\partial h_1}{\partial net_{h_1}} = h_1(1 - h_1) \approx 0.241$$

$$\frac{\partial net_{h_1}}{\partial w_1} = x_1 = 0.05$$

Therefore,

$$\frac{\partial E}{\partial w_1} = (0.0365)(0.241)(0.05)$$

$$\boxed{\frac{\partial E}{\partial w_1} \approx 0.000440}$$

Gradient for w_2: Follow the Connection



x_2 enters h_1 through w_2.

Chain Rule

$$\frac{\partial E}{\partial w_2} = \frac{\partial E}{\partial h_1} \frac{\partial h_1}{\partial net_{h_1}} \frac{\partial net_{h_1}}{\partial w_2}$$

$$= \delta_{h_1} \cdot h_1(1 - h_1) \cdot x_2$$

Computing the Gradient for w_2

We already know:

$$\delta_{h_1} = \frac{\partial E}{\partial h_1} \approx 0.0365$$

$$h_1(1 - h_1) = 0.5932(1 - 0.5932) \approx 0.241$$

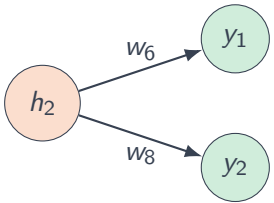
$$x_2 = 0.10$$

Therefore,

$$\frac{\partial E}{\partial w_2} = (0.0365)(0.241)(0.10)$$

$$\boxed{\frac{\partial E}{\partial w_2} \approx 0.00088}$$

Backpropagation to h_2



Key Idea

h_2 influences both outputs.
Therefore, its error signal receives contributions from **both paths**.

$$\delta_{h_2} = \text{feedback from } y_1 + \text{feedback from } y_2$$

Error Signal at Hidden Neuron h_2

Contribution Through y_1

$$(y_1 - T_1)y_1(1 - y_1)w_6$$

Contribution Through y_2

$$(y_2 - T_2)y_2(1 - y_2)w_8$$

Therefore,

$$\boxed{\delta_{h_2} = (y_1 - T_1)y_1(1 - y_1)w_6 + (y_2 - T_2)y_2(1 - y_2)w_8}$$

Computing δ_{h_2}

Path Through y_1

$$(0.7513 - 0.01)(0.7513)(1 - 0.7513)(0.45) \approx 0.0620$$

Path Through y_2

$$(0.773 - 0.99)(0.773)(1 - 0.773)(0.55) \approx -0.0209$$

$$\boxed{\delta_{h_2} \approx 0.0620 - 0.0209 = 0.0411}$$

Important

The hidden neuron receives error from **both outputs**.

Gradient for w_3



x_1 enters h_2 through w_3.

Chain Rule

$$\frac{\partial E}{\partial w_3} = \delta_{h_2} \cdot h_2(1 - h_2) \cdot x_1$$

$$= (0.0411)(0.5968)(1 - 0.5968)(0.05)$$

$$\boxed{\frac{\partial E}{\partial w_3} \approx 0.00049}$$

Gradient for w_4



x_2 enters h_2 through w_4.

Chain Rule

$$\frac{\partial E}{\partial w_4} = \delta_{h_2} \cdot h_2(1 - h_2) \cdot x_2$$

$$= (0.0411)(0.5968)(1 - 0.5968)(0.10)$$

$$\boxed{\frac{\partial E}{\partial w_4} \approx 0.00099}$$

All Weight Gradients: What Did Backpropagation Find?

Weight	Connection	Gradient
w_5	$h_1 \rightarrow y_1$	$+0.082$
w_6	$h_2 \rightarrow y_1$	$+0.083$
w_7	$h_1 \rightarrow y_2$	-0.023
w_8	$h_2 \rightarrow y_2$	-0.023
w_1	$x_1 \rightarrow h_1$	$+0.00044$
w_2	$x_2 \rightarrow h_1$	$+0.00088$
w_3	$x_1 \rightarrow h_2$	$+0.00049$
w_4	$x_2 \rightarrow h_2$	$+0.00099$

What Do the Signs Tell Us?

$+$ gradient $\Rightarrow$ decrease weight

$-$ gradient $\Rightarrow$ increase weight

Weight Update: Learning From the Error

Gradient Descent Rule

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial E}{\partial w}$$

Weight	Gradient	New Weight
w_1	$+0.00044$	0.14978
w_2	$+0.00088$	0.19956
w_3	$+0.00049$	0.24976
w_4	$+0.00099$	0.29951
w_5	$+0.082$	0.359
w_6	$+0.083$	0.359
w_7	-0.023	0.512
w_8	-0.023	0.562

Example: Updating w_7

$$w_7 = 0.50$$

$$\frac{\partial E}{\partial w_7} \approx -0.023$$

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial E}{\partial w}$$

$$w_7^{new} = 0.50 - (0.5)(-0.023)$$

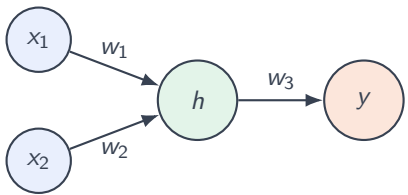
$$\boxed{w_7^{new} \approx 0.5115}$$

Why Did w_7 Increase?

The gradient is negative, so subtracting it causes the weight to increase.

Numerical Practice: Neural Network with ReLU

Network



Given

$x_1 = 2, \quad x_2 = -1$

$w_1 = 0.5, \quad w_2 = 0.4$

$b_h = -0.2$

$w_3 = 0.8$

$b_y = 0.1$

$T = 1$

$\eta = 0.1$

Activation Function

$$\text{ReLU}(z) = \max(0, z)$$

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z < 0 \end{cases}$$

Practice 1: Forward Pass

Step 1: Hidden Neuron

Calculate:

$$net_h = w_1 x_1 + w_2 x_2 + b_h$$

$$net_h = (0.5)(2) + (0.4)(-1) - 0.2$$

$$\boxed{net_h = ?}$$

Step 2: Apply ReLU

$$h = \text{ReLU}(net_h)$$

Practice 2: Calculate the Prediction

Output Neuron

Calculate:

$$net_y = w_3 h + b_y$$

Then apply ReLU:

$$y = \text{ReLU}(net_y)$$

$$\boxed{h \rightarrow net_y \rightarrow y}$$

Question

Is the prediction equal to the target $T = 1$?

Practice 3: How Wrong Is the Network?

Squared Error

$$E = \frac{1}{2}(T - y)^2$$

Substitute your prediction:

$$\boxed{E = \frac{1}{2}(1 - y)^2}$$

Question

Is the error large or small?
What does this tell us about the current weights?

Practice 4: Backpropagation Through ReLU

Output Error Signal

$$\delta_y = \frac{\partial E}{\partial y}\frac{\partial y}{\partial net_y}$$

$$\frac{\partial E}{\partial y} = y - T$$

$$\boxed{\delta_y = (y - T)\,\mathrm{ReLU}'(net_y)}$$

Question

What is $\mathrm{ReLU}'(net_y)$?
Therefore, calculate δ_y.

Practice 5: Gradient for Output Weight

The output neuron receives input h through w_3.

Chain Rule

$$\frac{\partial E}{\partial w_3} = \delta_y \frac{\partial net_y}{\partial w_3}$$

$$net_y = w_3 h + b_y$$

$$\boxed{\frac{\partial E}{\partial w_3} = \delta_y h}$$

Task

Calculate $\frac{\partial E}{\partial w_3}$.

Practice 6: Backpropagation to Hidden Neuron

The hidden neuron affects the output through w_3.

Hidden Error Signal

$$\delta_h = \delta_y \cdot w_3 \cdot \text{ReLU}'(net_h)$$

Question

$$\boxed{\delta_h = ?}$$

$$\boxed{\frac{\partial E}{\partial w_1} = \delta_h x_1}$$

$$\boxed{\frac{\partial E}{\partial w_2} = \delta_h x_2}$$

Practice 7: Update the Weights

$$\boxed{w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial E}{\partial w}}$$

$$\eta = 0.1$$

$$w_1^{new} = w_1 - 0.1 \frac{\partial E}{\partial w_1}$$

$$w_2^{new} = w_2 - 0.1 \frac{\partial E}{\partial w_2}$$

$$w_3^{new} = w_3 - 0.1 \frac{\partial E}{\partial w_3}$$

Final Question

After the update, should the error become larger or smaller? Why?

Challenge: What If the ReLU Neuron Dies?

Change only the hidden-layer bias:

$$\boxed{b_h = -1.5}$$

Recalculate

$$net_h = w_1 x_1 + w_2 x_2 + b_h$$

$$h = \mathrm{ReLU}(net_h)$$

$$\mathrm{ReLU}'(net_h) = ?$$

Challenge: What If the ReLU Neuron Dies?

Think Carefully

If

$$\text{ReLU}'(net_h) = 0$$

what happens to:

$$\frac{\partial E}{\partial w_1}, \qquad \frac{\partial E}{\partial w_2}?$$

Can this neuron learn?

Softplus: A Smooth ReLU

Function

$$\boxed{f(x) = \ln(1 + e^x)}$$

Derivative

$$\boxed{f'(x) = \frac{1}{1 + e^{-x}}}$$

$$\boxed{f'(x) = \sigma(x)}$$

ELU: Exponential Linear Unit

Function

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

Derivative

$$f'(x) = \begin{cases} 1, & x > 0 \\ \alpha e^x, & x \leq 0 \end{cases}$$

Why ELU?

Negative inputs produce a smooth negative output instead of becoming exactly zero.

SELU: Scaled Exponential Linear Unit

Function

$$f(x) = \lambda \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

Derivative

$$f'(x) = \lambda \begin{cases} 1, & x > 0 \\ \alpha e^x, & x \leq 0 \end{cases}$$

Key Idea

SELU is designed to encourage **self-normalizing neural networks**.

Leaky ReLU: Keep a Small Gradient

Function

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

Usually,

$$\alpha \ll 1$$

Derivative

$$f'(x) = \begin{cases} 1, & x > 0 \\ \alpha, & x < 0 \end{cases}$$

PReLU: Let the Network Learn the Slope

Function

$$\boxed{f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}}$$

Derivative

$$f'(x) = \begin{cases} 1, & x > 0 \\ \alpha, & x < 0 \end{cases}$$

Leaky ReLU: α is fixed; PReLU: α is learned

GELU: Gaussian Error Linear Unit

Definition

$$\boxed{\text{GELU}(x) = x\Phi(x)}$$

where $\Phi(x)$ is the standard normal CDF.

Derivative

$$\boxed{\text{GELU}'(x) = \Phi(x) + x\phi(x)}$$

where $\phi(x)$ is the standard normal PDF.

Intuition

GELU smoothly gates the input according to its magnitude instead of using a hard cutoff like ReLU.

SiLU / Swish: Self-Gated Activation

Function

$$\boxed{f(x) = x\sigma(x)}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Derivative

$$\boxed{f'(x) = \sigma(x) + x\sigma(x)(1 - \sigma(x))}$$

Intuition

The input controls its own gate through the sigmoid.

Mish: Smooth Self-Gated Activation

Function

$$f(x) = x \tanh(\text{Softplus}(x))$$

Derivative

$$f'(x) = \tanh(s) + x\,\sigma(x)\left[1 - \tanh^2(s)\right]$$

Intuition

Mish is smooth, non-monotonic, and self-gated.

Hard Sigmoid

Function

$$f(x) = \begin{cases} 0, & x \leq -2.5 \\ 0.2x + 0.5, & -2.5 < x < 2.5 \\ 1, & x \geq 2.5 \end{cases}$$

Derivative

$$f'(x) = \begin{cases} 0, & x < -2.5 \\ 0.2, & -2.5 < x < 2.5 \\ 0, & x > 2.5 \end{cases}$$

Softmax: From Scores to Probabilities

Function

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Properties

$$0 \leq p_i \leq 1$$

$$\sum_{i=1}^{K} p_i = 1$$

Softmax Derivative

For

$$p_i = \text{softmax}(z_i)$$

the derivative is

$$\boxed{\frac{\partial p_i}{\partial z_j} = p_i(\delta_{ij} - p_j)}$$

Therefore,

$$\frac{\partial p_i}{\partial z_i} = p_i(1 - p_i)$$

Softmax Derivative

while for $i \neq j$,

$$\frac{\partial p_i}{\partial z_j} = -p_i p_j$$

Important

Unlike Sigmoid, Softmax outputs are coupled: changing one score affects all class probabilities.

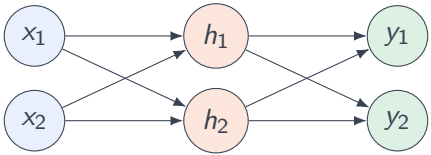
Which Activation Should We Use?

Activation	Main Characteristic	Typical Use
Sigmoid	Smooth, bounded $(0, 1)$	Binary output
Tanh	Zero-centered	Older hidden layers
ReLU	Simple, fast	General deep networks
Leaky ReLU	Keeps negative gradient	Avoid dead neurons
PReLU	Learnable negative slope	Deep CNNs
ELU	Smooth negative region	Deep networks
SELU	Self-normalizing	Specific architectures
Softplus	Smooth ReLU	Smooth optimization
GELU	Smooth probabilistic gating	Transformers
SiLU/Swish	Self-gated	Modern deep networks
Mish	Smooth, non-monotonic	Modern architectures
Softmax	Probability distribution	Multiclass output

Which Activation Should We Use?

There is no universally best activation function.
The choice depends on the architecture, optimization behavior, and task.

From Neural Network to Python



In Python, We Store the Connections as Matrices

$\boxed{W_1} \longrightarrow$ Input $\rightarrow$ Hidden weights

$\boxed{W_2} \longrightarrow$ Hidden $\rightarrow$ Output weights

Key Idea

The code is implementing the same network we just calculated by hand.

Step 1: Activation Function

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))
```

What Does It Do?

Converts a neuron's weighted sum into an output between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Connect to Our Example

Every hidden and output neuron uses the sigmoid activation.

Step 2: Create the Network

```
class DNN:
    def __init__(self, input_size,
                 hidden_size, output_size):
        self.W1 = np.random.randn(
            input_size, hidden_size)
        self.b1 = np.zeros(hidden_size)
        self.W2 = np.random.randn(
            hidden_size, output_size)
        self.b2 = np.zeros(output_size)
```

What Are We Creating?

$$W_1,\ b_1 \quad \Rightarrow \quad \text{Input} \rightarrow \text{Hidden}$$

$$W_2,\ b_2 \quad \Rightarrow \quad \text{Hidden} \rightarrow \text{Output}$$

What Do W_1 and W_2 Contain?

$$W_1 = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} w_5 & w_7 \\ w_6 & w_8 \end{bmatrix}$$

Meaning

W_1 : Input neurons $\rightarrow$ Hidden neurons

W_2 : Hidden neurons $\rightarrow$ Output neurons

Step 3: Forward Pass

```
def forward(self, X):
    self.z1 = np.dot(X, self.W1) + self.b1
    self.a1 = sigmoid(self.z1)
    self.z2 = np.dot(self.a1, self.W2) + self.b2
    self.a2 = sigmoid(self.z2)
    return self.a2
```

What Do z and a Mean?

z = weighted sum before activation

a = activated output

Step 4: Measure the Error

Mean Squared Error

$$L = \frac{1}{2}(y - \hat{y})^2$$

Prediction $\longleftrightarrow$ Target

Goal

A good network should make L as small as possible.

Our Learning Cycle

Predict $\rightarrow$ Measure Error $\rightarrow$ Correct Weights

Step 5: Backpropagation at the Output

```
dz2 = (self.a2 - y) * \
      (self.a2 * (1 - self.a2))

dW2 = np.dot(self.a1.T, dz2)

db2 = np.sum(dz2, axis=0)
```

Connect to Our Derivation

$$\boxed{\delta_2 = (a_2 - y)\, a_2(1 - a_2)}$$

This is the same idea as:

$$(y - T)\, y(1 - y)$$

Step 6: Send the Error Backward

```
dz1 = np.dot(dz2, self.W2.T) \
      * (self.a1 * (1 - self.a1))

dW1 = np.dot(X.T, dz1)

db1 = np.sum(dz1, axis=0)
```

What Is Happening?

$$\boxed{\text{Output error} \rightarrow W_2 \rightarrow \text{Hidden error}}$$

Then the hidden layer applies its sigmoid derivative:

$$a_1(1 - a_1)$$

Why Does the Code Use Matrix Multiplication?

For one neuron:

$$z = w_1 x_1 + w_2 x_2 + b$$

For many neurons, we perform all weighted sums together:

$$\boxed{Z = XW + b}$$

One equation replaces many individual calculations.

Key Idea

Matrix operations allow us to calculate many neuron connections simultaneously.

Step 7: Update the Weights

```
self.W2 -= lr * dW2
self.b2 -= lr * db2

self.W1 -= lr * dW1
self.b1 -= lr * db1
```

Gradient Descent

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W}$$

Remember

We move in the direction that **reduces the error**.

Step 8: Repeat the Learning Cycle

```
def train(self, X, y, epochs, lr):

    for _ in range(epochs):

        output = self.forward(X)

        self.backward(X, y, lr)
```

Forward → Error → Backward → Update

⇓

Repeat for many epochs

Step 9: Making a Prediction

```
def predict(self, X):

    output = self.forward(X)

    return np.round(output)
```

What Happens?

$$X \rightarrow \text{Forward Pass} \rightarrow \text{Output Probability}$$

$$\boxed{\text{Probability} \rightarrow \text{Class Label}}$$

If output $= 0.82$, rounding gives class 1. If output $= 0.21$, rounding gives class 0.